CARE-AI

V1 Architecture & Build Plan

SLM-first longitudinal patient intelligence and early-warning prototype

A practical first version optimized for local development, lower inference cost, traceable evidence, and human oversight.

IMPORTANT SAFETY BOUNDARY

This is an educational engineering prototype using synthetic/demo data. It is not clinically validated, does not diagnose or treat, and must not be used to make real clinical decisions. The 0–100 signal index is a rule-based demo value, not a clinical risk probability.

Scope: First working slice • Stack: FastAPI + SQLite + static dashboard • AI: optional local SLM, optional isolated LLM narration

Version 1.0 | October 2026

Executive summary

The source blueprint describes longitudinal patient-state monitoring, specialized analysis, evidence-backed warnings, uncertainty handling, auditability, and clinician decision authority. V1 preserves these principles and trims the infrastructure to a testable local system.

CARE-AI should not be built as one large language-model prompt. It should assemble a timestamped patient timeline, calculate personal baselines, detect meaningful changes with normal code, extract note phrases only when needed, link each finding to evidence, and let a separate safety gate decide whether the software should monitor, request review, or abstain.

V1 success path

Step	Stage	V1 behavior
1	Ingest	Validate a vital/lab measurement or a synthetic note.
2	Compare	Build the timeline and personal baseline; calculate deltas and recent trends.
3	Extract	Use a local SLM for explicit note mentions only when configured; otherwise use deterministic phrase matching.
4	Score	Calculate a documented demonstration index; never claim clinical probability without a validated model.
5	Gate	Attach evidence and apply a deterministic data-quality/safety check.
6	Review	Display the result and let a human record a review outcome; audit each workflow.

Non-negotiable principles

- Deterministic core, probabilistic text extraction: never call an LLM to calculate a baseline, compare two values, validate a unit, or persist a record.
- Source-grounded evidence: measurements remain in the database; extracted phrases must be found in the source note or be discarded.
- Safety can abstain: absent, inadequate, inconsistent or failed inputs must not result in an invented score.
- Human decision authority: the UI is a decision-support prototype only; it does not prescribe, diagnose or autonomously act.
- No unsupported validation claims: there is no labeled clinical outcome dataset included, so V1 does not provide a trained clinical predictor.

What to build in V1

One modular monolith is deliberate: fewer moving parts, lower cost, local operation, and simpler debugging while the end-to-end data flow is proved.

Included in V1	Deferred until justified
Patient labels, synthetic observations, notes, assessments, reviews and audit records	Production identity provider, SSO and multi-organization access
SQLite source of truth, FastAPI/Pydantic validation, OpenAPI docs	PostgreSQL, managed object storage and event streaming
Personal baseline, trend/change rules and transparent demo signal index	Trained forecasting model; LSTM/Transformer; model registry
Local Ollama SLM adapter and deterministic fallback for note extraction	Multi-model agent framework, multiple deployed services and GPU fleet
Evidence builder and deterministic safety/abstention gate	Vector DB / RAG, clinical guideline ingestion, OCR, medical imaging
Static dashboard served directly by FastAPI; local scripts, tests and optional Docker	Cloud production operations, FHIR integration, prospective validation

Cost-first technology stack

Layer	V1 choice	Reason
UI	HTML + CSS + vanilla JavaScript, served by FastAPI	No separate Node build or frontend server required
API / application	Python 3.11+ + FastAPI + Pydantic	Typed validation, API docs, one service
Persistence	SQLite	Zero separate database server for local prototype
Numeric reasoning	Python statistics + transparent rules	No token cost, easy to audit and unit test
SLM	Ollama-compatible local model, configurable	Optional local note phrase extraction; no cloud call by default
LLM	OpenAI-compatible adapter; off by default	Wording-only helper, not allowed to set score/status
QA / packaging	pytest, Docker optional	Repeatable local checks, simple deployment

WHY NOT LANGGRAPH / VECTOR DB / TRANSFORMER YET?

The first milestone should validate the ingestion-to-review flow. Add orchestration frameworks, semantic retrieval and temporal neural models only when measured workflow complexity, an approved evidence collection, or labeled data justify the extra cost and operational burden.

SLM-first, LLM-optional design

The lowest-cost model call is the one not made. Keep numerical and safety operations in ordinary code; use language models only for constrained text tasks.

Task	Primary mechanism	Model policy / guardrail
Schema validation, unit checks, timestamps, duplicate prevention	Pydantic + Python	No model call
History, timeline and patient baseline	SQLite queries + Python median/delta/trend	No model call
Significant-change gate	Deterministic rules	No model call; log reasons
Demo signal index	Versioned rule weights	Not a clinical probability; no LLM override
Note phrase extraction	Local Ollama SLM when configured	JSON schema; source-grounding checks; deterministic fallback on error
Evidence assembly	Source IDs and exact note phrase references	No model-generated source facts
Safety gate	Deterministic minimum-data and validity checks	Can abstain even if a score was tentatively calculated
Readable summary	Deterministic template	Optional larger LLM only rephrases de-identified metadata

Model integration contracts

- SLM returns structured mention objects, not diagnoses or treatment recommendations. Output that cannot be grounded in the note is rejected.
- If Ollama is missing, offline or times out, the backend continues with deterministic extraction and exposes `rules` or `fallback` metadata.
- LLM narration is disabled by default. When deliberately enabled, the request contains only counts, score, state and evidence types—not patient IDs, names, raw notes or exact note text.
- The LLM response can only replace wording. It cannot change the numeric index, status, evidence, safety reasons or reviewer outcome.
- Never commit `.env` or API keys. Hosted inference may incur cost and needs provider privacy review before use.

V1 logical flow

Dashboard	**API + validation**	**Timeline + baseline**	**Change detection**
Optional local SLM	**Demo rules**	**Evidence builder**	**Safety gate**
Template / optional LLM wording	**Audit + human review**	**SQLite source of truth**	

Model output never bypasses the evidence builder or safety gate.

Observation-to-review sequence

Each run should be reproducible from timestamped input records and a versioned ruleset. In V1, agent boundaries are code modules inside one backend service.

1. Ingest and persist Validate the type, canonical unit, numeric demo input bounds, timezone-aware timestamp and source event ID; write the event before analysis and add an audit event.

2. Build patient state Load the chronological timeline; group by observation type; compute a baseline from prior observations and preserve observation time separately from ingestion time.

3. Detect change Calculate baseline deviation and trend rules. Do not call a model if a language task is absent or a numeric calculation is sufficient.

4. Extract note mentions if present Invoke the configured local SLM, constrain the output schema and ground extracted phrases in source text. If unavailable or invalid, use deterministic fallback and disclose it.

5. Compute demo signal index Apply the documented example weights. Missing measurements are unknown, never silently treated as normal. Score rules and version are returned as metadata.

6. Build evidence and apply safety Link each evidence item to an observation ID or note ID/phrase. Data-quality checks can return `ABSTAIN`; otherwise demo flow returns `MONITOR` or `REVIEW`.

7. Generate wording and persist review Use template wording by default. Optional LLM can rephrase metadata but cannot change structured fields. Persist the assessment, audit record and any human review outcome.

API surface

Endpoint	Purpose
GET `/api/health`	App readiness + safe configuration flags
GET / POST `/api/patients`	List/create synthetic or de-identified patient labels
POST `/api/demo/seed`	Idempotently seed the demo record
GET `/api/patients/{id}/timeline`	Chronological observations and notes
POST `/api/patients/{id}/observations`	Validate and store a measurement
POST `/api/patients/{id}/notes`	Store note and return extraction metadata
POST `/api/patients/{id}/analyze`	Build signal index, evidence and safety result
GET `/api/patients/{id}/analyses`	Assessment history
POST `/api/analyses/{id}/review`	Save a human review outcome
GET `/api/patients/{id}/audit`	Latest audit events

Data model, evidence and failure states

Entity	Key fields	Purpose
Patient	id, display label, created_at, data_kind	Internal identity for demo records; not a real-world identifier
Observation	id, patient_id, type, value, unit, observed_at, ingested_at, source, quality flag, event ID	Longitudinal measurement record; duplicate source IDs rejected
Clinical note	id, patient_id, text, created_at, extraction mode, structured mentions	Note phrase extraction with source context; use only synthetic/de-identified text
Analysis	id, patient_id, nullable signal_index, state, summary, evidence JSON, safety JSON, model/rule metadata	Reproducible assessment trace
Review	analysis ID, reviewer label, outcome, comment, timestamp	Human review only; no automatic medical action
Audit event	action, component, status, timestamp, compact details	Traceability without dumping raw notes or secrets into logs

Safety states

State	Meaning in V1	UI behavior
MONITOR	Illustrative demo rules did not request review and data gate passed	Show the evidence; do not claim health or no risk
REVIEW	A demo rule/change requested human review	Highlight a workflow review, not an actual clinical alert
ABSTAIN	Insufficient evidence or required processing was not safe/available	Do not display numeric index as usable; show reason codes

Security and privacy boundary

- Default dataset is synthetic; use approved de-identified data only for prototype testing.
- This V1 includes no production authentication or role-based access. Do not deploy to the public internet or use as a real patient record system.
- Cap note length and validate all inputs. Keep CORS limited to local origins, and never expose secret values from health/config endpoints.
- Avoid raw note text in audit events. Local SQLite is the source of truth; back up files before reset or migration.
- Clinical validation requires a defined outcome, appropriate labeled data, patient/time-aware splits, calibration, subgroup evaluation, prospective review, governance and qualified oversight.

ABSTENTION RULE

If the input is too sparse, the model fails, or evidence cannot be grounded, return a transparent abstention or deterministic fallback. Never turn a timeout, missing value or low confidence into a fabricated prediction.

What the ZIP contains

```
CARE-AI-V1/
├── backend/
│   ├── app/
│   │   ├── agents/        monitoring, vitals, note NLP, evidence, safety
│   │   ├── services/      optional LLM wording adapter
│   │   ├── db.py          SQLite schema + connection boundary
│   │   ├── schemas.py     request validation and canonical units
│   │   └── main.py        API routes + dashboard hosting
│   ├── tests/             unit/API tests
│   ├── requirements.txt
│   └── .env.example
├── frontend/              index.html, app.js, styles.css
├── docs/                  architecture, data dictionary, run guide
├── infra/                 backend Dockerfile + Compose
├── BUILD_PROMPT.md        copy-paste coding-agent specification
└── README.md
```

Run locally on Windows (PowerShell)

Step	Command
1. Go to backend	cd backend
2. Create environment	py -m venv .venv
3. Activate environment	.\.venv\Scripts\Activate.ps1
4. Install dependencies	python -m pip install --upgrade pip ; pip install -r requirements.txt
5. Configure local settings	Copy-Item .env.example .env
6. Start the app	uvicorn app.main:app --reload --host 127.0.0.1 --port 8000

Open the dashboard at http://127.0.0.1:8000 and API docs at http://127.0.0.1:8000/docs. The first run creates the local SQLite database automatically. Full Windows/macOS/Linux, optional Ollama, optional LLM and Docker instructions are in `docs/RUN_GUIDE.md`.

Enable local SLM (optional)

Install Ollama, pull a small model such as `qwen3.5:0.8b`, then set `CARE_AI_SLM_MODE=ollama`, `CARE_AI_OLLAMA_BASE_URL=http://localhost:11434`, and `CARE_AI_SLM_MODEL=qwen3.5:0.8b` in `backend/.env`. Restart the backend. If the local service is unavailable, deterministic extraction remains available.

Acceptance tests and staged roadmap

- App starts without an API key or language model installed.
- Dashboard and `/docs` load; `/api/health` returns `ok`.
- Repeated demo seeding does not duplicate example observations.
- Validated observation -> timeline -> baseline/change detection -> analysis -> evidence -> safety state works.
- Insufficient data produces `ABSTAIN` instead of a fabricated score.
- Ollama offline path falls back to deterministic extraction and returns metadata.
- Optional LLM is disabled by default and cannot change index/status/evidence.
- Review outcomes persist and appear in audit history.
- Automated tests pass with `pytest -q` from `backend/`.

Roadmap after the first working slice

Phase	Build next	Gate before proceeding
V1.1	Authentication/RBAC, better DB abstraction, error monitoring and retention policy	Threat model, access tests, no-secret logging
V1.2	Choose one clinical outcome; obtain suitable approved dataset and define labels	Data governance + evaluation protocol accepted by qualified reviewers
V1.3	Train and benchmark a classical baseline (e.g. Logistic Regression/XGBoost)	Patient/time-aware split, calibration, sensitivity, false-alert review and subgroup analyses
V1.4	Evaluate clinical NLP/SLM extraction, evidence retrieval and temporal models	Demonstrated value over deterministic baselines; documented cost and latency
V2 / production study	PostgreSQL, FHIR, queues, formal model registry, prospective evaluation	Security, privacy, clinical workflow and regulatory review complete

FINAL PRINCIPLE

Build the early-warning workflow, not an autonomous doctor. Keep the SLM small and local for narrow language tasks; keep numerical reasoning, evidence provenance, and the safety gate deterministic and independently testable.

Source alignment: this V1 architecture is a cost-optimized implementation plan derived from the provided CARE-AI technical architecture. It preserves the original longitudinal approach, separated agent responsibilities, source-linked evidence, safety abstention, audit trail and clinician oversight while trimming V1 infrastructure.